

ECE 315

Steven Knudsen, PhD, PEng

Lecture 10 :

- Mutexes and Semaphores example
- Synchronization
- Priority Inversion
- Deadlock



**UNIVERSITY
OF ALBERTA**

Last Lecture

- Quiz #1
 - General Q&A
- Before that
- More on RTOS
 - Tasks and Scheduling
 - Protecting Critical Code

This Lecture

- Mutexes and Semaphore example
- Synchronization
- Priority Inversion
- Deadlock

Learning Objectives

- Understand what semaphore and mutexes are used for
- Know how to set up a mutex in FreeRTOS
- Understand the condition of "deadlock" and the conditions that must be true for it to happen
- Know some strategies to avoid deadlock
- Be able to define "priority inversion"
- Explain the FreeRTOS mechanism that resolves priority inversion
- Define watchdog timer

Protection of Critical Sections – Semaphores & Mutexes

- **Mutexes** and **semaphores** are *synchronization* primitives
 - are tools used in multitasking embedded RTOSes
 - coordinate access to shared resources and control task execution order
- A **semaphore** is a counter-based synchronization object that controls access to a shared resource by counting available slots or permissions.
- A **mutex** is like a binary semaphore but designed specifically for mutual exclusion – ensuring only one task at a time can access a shared resource (like a UART, sensor, or global variable)

Without synchronization, race conditions, data corruption, or deadlocks can occur.

Mutexes and Semaphores

Typical Use of a Mutex

1. Create a mutex at system startup.
2. Before a task uses the shared resource, it takes (locks) the mutex.
3. After using the resource, it releases (unlocks) the mutex.
4. If another task tries to lock it while it's in use, it blocks until the mutex becomes available.

Typical Use of a Semaphore

1. One task (or ISR) "gives" the semaphore to signal an event.
2. Another task "takes" the semaphore and continues when the signal is received.
3. Often used for event notification or task synchronization.

FreeRTOS Mutex Example

```
#include "FreeRTOS.h"
#include "task.h"
#include "semphr.h"

SemaphoreHandle_t xUARTMutex;

void vTask1(void *pvParameters) {
    for(;;) {
        if (xSemaphoreTake(xUARTMutex, portMAX_DELAY) == pdTRUE) {
            printf("Task 1 writing to UART...\n");
            vTaskDelay(pdMS_TO_TICKS(500));
            xSemaphoreGive(xUARTMutex);
        }
        vTaskDelay(pdMS_TO_TICKS(100));
    }
}

void vTask2(void *pvParameters) {
    for(;;) {
        if (xSemaphoreTake(xUARTMutex, portMAX_DELAY) == pdTRUE) {
            printf("Task 2 writing to UART...\n");
            vTaskDelay(pdMS_TO_TICKS(300));
            xSemaphoreGive(xUARTMutex);
        }
        vTaskDelay(pdMS_TO_TICKS(100));
    }
}
```

FreeRTOS Mutex Example

```
int main(void) {  
    // Create the mutex before starting tasks  
    xUARTMutex = xSemaphoreCreateMutex();  
  
    if (xUARTMutex != NULL) {  
        xTaskCreate(vTask1, "Task1", 128, NULL, 1, NULL);  
        xTaskCreate(vTask2, "Task2", 128, NULL, 1, NULL);  
        vTaskStartScheduler();  
    }  
  
    for(;;); // Should never reach here  
}
```


FreeRTOS Mutex Example

Key Components:

- Mutex Creation: `xSemaphoreCreateMutex()` creates the mutex before tasks start
- Taking the Mutex: `xSemaphoreTake(xMutex, portMAX_DELAY)` locks the mutex
- Giving the Mutex: `xSemaphoreGive(xMutex)` unlocks the mutex
- Shared Resource: Both tasks safely increment `sharedCounter`

How it Works:

- Both tasks try to increment the same counter
- Before accessing the counter, each task must acquire the mutex
- If one task holds the mutex, the other waits until it's released
- This prevents race conditions and data corruption

```
#include "FreeRTOS.h"
#include "task.h"
#include "semphr.h"
#include <stdio.h>

// Shared resource (protected by mutex)
static int sharedCounter = 0;

// Mutex handle
SemaphoreHandle_t xMutex;
```



FreeRTOS Mutex Example

```
/* Task 1 - Increments the shared counter */
void vTask1(void *pvParameters)
{
    int localCount = 0;
    while(1) {
        // Take the mutex before accessing shared resource
        // Wait indefinitely (portMAX_DELAY) until mutex is available
        if(xSemaphoreTake(xMutex, portMAX_DELAY) == pdTRUE) {
            // Critical section - safely access shared resource
            sharedCounter++;
            localCount = sharedCounter;
            printf("Task 1: Counter = %d\n", localCount);

            // Simulate some work
            vTaskDelay(pdMS_TO_TICKS(100));

            // Give back the mutex
            xSemaphoreGive(xMutex);
        } // if

        // Do other work outside critical section
        vTaskDelay(pdMS_TO_TICKS(500));
    } // while
}
```

FreeRTOS Mutex Example

```
/* Task 2 - Also increments the shared counter */
void vTask2(void *pvParameters)
{
    int localCount = 0;

    while(1) {
        // Take the mutex before accessing shared resource
        if(xSemaphoreTake(xMutex, portMAX_DELAY) == pdTRUE) {
            // Critical section - safely access shared resource
            sharedCounter++;
            localCount = sharedCounter;
            printf("Task 2: Counter = %d\n", localCount);

            // Simulate some work
            vTaskDelay(pdMS_TO_TICKS(150));

            // Give back the mutex
            xSemaphoreGive(xMutex);
        } // if

        // Do other work outside critical section
        vTaskDelay(pdMS_TO_TICKS(700));
    } // while
}
```

FreeRTOS Mutex Example

```
/* Main function - sets up tasks and starts scheduler */
int main(void) {
    // Create the mutex before creating tasks
    xMutex = xSemaphoreCreateMutex();
    if(xMutex != NULL) {
        // Create Task 1
        xTaskCreate(
            vTask1,           // Task function
            "Task1",         // Task name
            1000,             // Stack size (words)
            NULL,             // Task parameters
            1,               // Priority
            NULL              // Task handle
        );

        // Create Task 2
        xTaskCreate(vTask2, "Task2", 1000, NULL, 1, NULL );

        // Start the scheduler
        vTaskStartScheduler();
    } else {
        printf("Failed to create mutex!\n");
    }

    // Should never reach here
    while(1);

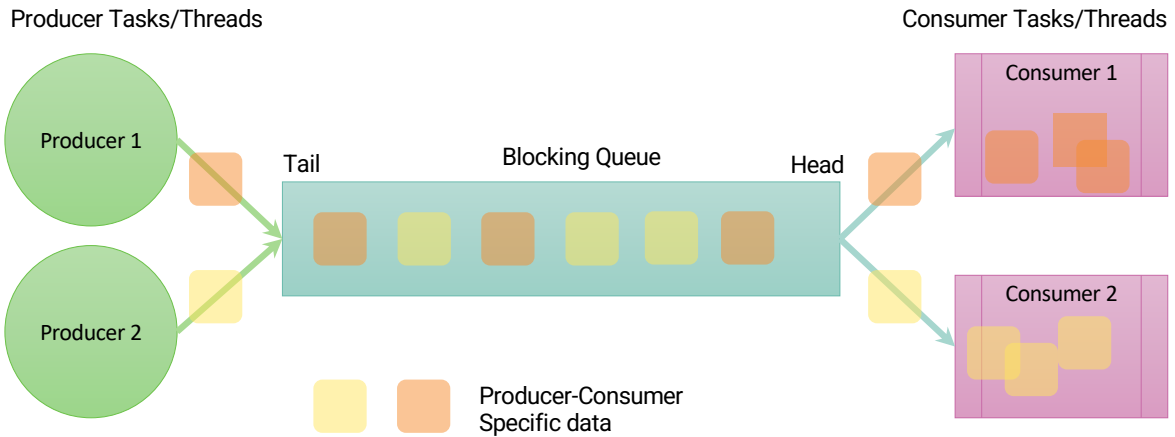
    return 0;
}
```

Synchronization via a Queue

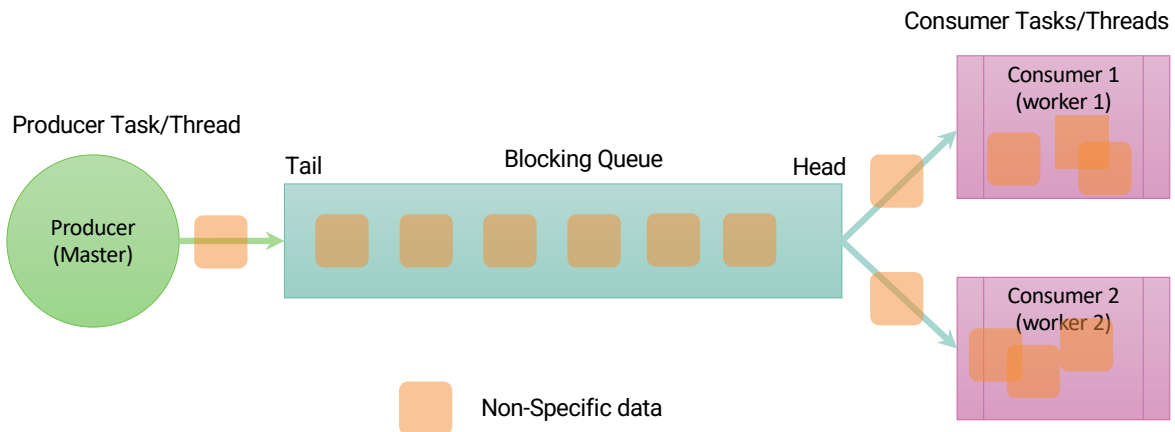
Tasks (e.g., one producer task and one consumer task) synchronize through the transfer of data.

- This is often conveniently implemented using a queue.
1. The producer task sends data to the consumer task by adding/writing data to the back end of a queue.
 2. The consumer task calls a blocking receive function that attempts to read data from the front of the queue.
 - If data is immediately available in the queue, then the consumer task can proceed and process the new data.
 - If there is no data present in the queue, then the consumer task is switched off the CPU and is added to the queue of tasks that are blocked on the queue.
 - Later, when data is added to an empty queue, the highest priority task that is blocked on the queue becomes ready-to-run.

Synchronization via a Queue; Producer-Consumer



Producer-Consumer aka Master-Worker



This is known as a programming pattern.

A pattern is a way of doing things that happens often, often enough that it applies to many problems. No point reinventing it...

ECE 315 Lec 5 FreeRTOS Example cont'd

```
/**
 * @brief Repeatedly flash the Pico's built-in LED.
 */
void led_task_consumer(void* unused_arg) {

    // This variable will take a copy of the value
    // added to the FreeRTOS xQueue
    uint8_t passed_value_buffer = 0;

    // Configure the Pico's on-board LED
    gpio_init(PICO_DEFAULT_LED_PIN);
    gpio_set_dir(PICO_DEFAULT_LED_PIN, GPIO_OUT);
```

```
    while (true) {
        // Check for an item in the FreeRTOS xQueue
        if (xQueueReceive(queue, &passed_value_buffer, portMAX_DELAY) == pdPASS) {
            // Received a value so flash the GPIO LED accordingly
            // (NOT the sent value)

            printf("%-9llms consume %d\n", get_ms_freertos(), passed_value_buffer);
            gpio_put(PICO_DEFAULT_LED_PIN, passed_value_buffer == 1 ? 0 : 1);
        }
    }
}
```


ECE 315 Lec 5 FreeRTOS Example cont'd

```
/**
 * @brief Repeatedly flash the Pico's built-in LED.
 */
void led_task_producer(void* unused_arg) {
    // Store the Pico LED state
    uint8_t pico_led_state = 0;

    while (true) {
        // Add the LED state to the FreeRTOS xQUEUE
        pico_led_state = !pico_led_state; // toggle LED state
        printf("%-9llms produce %d\n", get_ms_freertos(), pico_led_state);
        xQueueSendToBack(queue, &pico_led_state, 0);
        vTaskDelay(ms_delay);
    }
}
```

ECE 315 Lec 5 FreeRTOS Example cont'd

```
// Set up the event queue
queue = xQueueCreate(4, sizeof(uint8_t));
```

```
// Start the FreeRTOS scheduler
// Only proceed with valid tasks
```

```
if (prod_status == pdPASS || con_status == pdPASS || red_status == pdPASS || \
    green_status == pdPASS || blue_status == pdPASS || block_status == pdPASS) {
    vTaskStartScheduler();
}
```

```
// We should never get here, but just in case...
while(true) {
    // NOP
}
```

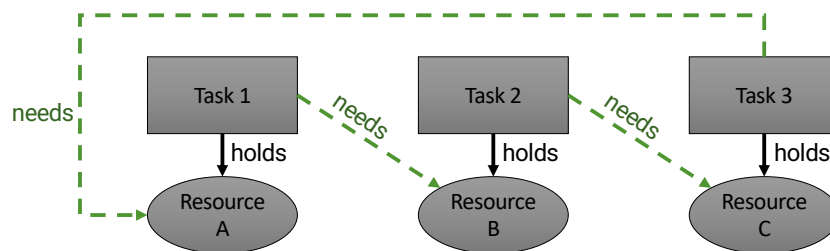
```
/*
 * RUNTIME START
 */
int main() {
    . . .

    // Set up the event queue
    queue = xQueueCreate(4, sizeof(uint8_t));

    . . .
} // main()
```

Deadlock

- Is a situation where two or more tasks are permanently blocked, each waiting for a resource (such as a mutex, semaphore, or message queue) that can only be released by another blocked task
- As a result, none of the tasks can proceed, and the system or part of it becomes frozen indefinitely



Deadlock

- Edward Coffman's conditions (which apply to general and embedded systems), a deadlock can occur if all four of the following are true:
 - **Mutual Exclusion** – Resources cannot be shared (only one task at a time can use them).
 - **Hold and Wait** – A task holds one resource while waiting for another.
 - **No Preemption** – Resources cannot be forcibly taken away.
 - **Circular Wait** – A circular chain exists where each task holds one resource and waits for another.
- Breaking any one of these conditions prevents deadlock.

Priority Inversion

- Related to Deadlock is the situation called *Priority Inversion*.
- It is a condition where a high-priority task is **indirectly** blocked by a lower-priority task, due to a shared resource (e.g., a mutex) being held by that lower-priority task
- As a result, a medium-priority task (that does not depend on the shared resource) can preempt the low-priority one — causing the high-priority task to wait even longer.
- This inverts the intended priority order: the low-priority task effectively delays a high-priority task.

How Priority Inversion Happens...

1. Low-priority task (L) *locks a shared resource* (e.g., a UART protected by a mutex).
2. High-priority task (H) becomes ready and *tries to lock the same resource* – but must wait because L holds it.
3. Medium-priority task (M) *that doesn't need the resource* becomes ready and, since it has higher priority than L, it preempts L.
4. L can't finish and release the resource, so *H remains blocked* – even though H has the highest priority.

The system's behavior now effectively inverts priorities:

H (highest) < M (medium) < L (lowest).

Solutions to Priority Inversion

- Complicated to find safe ways of forcing a low-priority task to give up exclusive access to a resource in a critical section.
- A common strategy for solving the priority inversion problem is to *temporarily raise the priority of the low-priority task to at least the priority of any high-priority task that gets blocked on the semaphore* that controls allocation of the contested resource. This allows the low-priority task to finish using its resources quickly so that the high-priority task can proceed. Medium-priority tasks cannot interfere.
- FreeRTOS provides **mutual exclusion** semaphores (**mutexes**), which are binary semaphores that cause the priority of the semaphore-holding task to be raised to that of any higher priority task that gets blocked on the same semaphore.

Watchdog Timers

- A watchdog timer (WDT) is a hardware or software timer used to detect and recover from software malfunctions,
 - e.g., such as one task hogging the CPU, multitasking deadlock, memory leak, etc.
- It acts like a safety mechanism that ensures the system keeps running properly.
- If the software fails to "pet" (reset or refresh) the watchdog timer within a specified time period, the watchdog assumes the system is stuck or malfunctioning and forces a system reset or other corrective action
- They can be dangerous – a faulty hardware or software WDT can result in a fast, infinite reset loop ("death loop") that is hard to debug

Next Lecture

- Computer Interfacing
- Overview of some interfaces
- Duplex
- The nature of analogue and digital signals

Questions

